Step 1: Install Libraries

```
!pip install transformers datasets torch scikit-learn -q
```

Step 2: Import Libraries

```
import torch
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments
from sklearn.metrics import accuracy_score
```

Step 3: Load Dataset

```
dataset = load_dataset("imdb")

train_data = dataset["train"].shuffle(seed=42).select(range(2000))
test_data = dataset["test"].shuffle(seed=42).select(range(1000))
```

```
README.md: 7.81k/? [00:00<00:00, 180kB/s]
plain_text/train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:01<00:00, 160MB/s]
plain_text/test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:00<00:00, 25.9MB/s]
plain_text/unsupervised-00000-of-00001.p(…): 100% 42.0M/42.0M [00:01<00:00, 98.7MB/s]
Generating train split: 100% 25000/25000 [00:01<00:00, 26285.69 examples/s]
Generating test split: 100% 25000/25000 [00:00<00:00, 42263.81 examples/s]
Generating unsupervised split: 100% 50000/50000 [00:01<00:00, 52077.98 examples/s]
```

Step 4: Load Tokenizer

```
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 998B/s]
vocab.txt: 232k/? [00:00<00:00, 2.60MB/s]
tokenizer.json: 466k/? [00:00<00:00, 1.09MB/s]
```

Step 5: Tokenize Text

```
def tokenize(example):
    return tokenizer(example["text"], padding="max_length", truncation=True)

train_data = train_data.map(tokenize, batched=True)
test_data = test_data.map(tokenize, batched=True)
```

```
Map: 100% 2000/2000 [00:09<00:00, 212.08 examples/s]
Map: 100% 1000/1000 [00:03<00:00, 322.67 examples/s]
```

Step 6: Convert to PyTorch Format

```
train_data.set_format(type="torch", columns=["input_ids","attention_mask","label"])
test_data.set_format(type="torch", columns=["input_ids","attention_mask","label"])
```

Step 7: Load Pretrained Model

```
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
```

```
    num_labels=2
)
```

```
Loading weights: 100%|| 100/100 [00:00<00:00, 197.12it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                     | Status     |
------------------------+------------+-
vocab_layer_norm.weight | UNEXPECTED |
vocab_transform.weight  | UNEXPECTED |
vocab_projector.bias    | UNEXPECTED |
vocab_transform.bias    | UNEXPECTED |
vocab_layer_norm.bias   | UNEXPECTED |
classifier.weight       | MISSING    |
pre_classifier.weight   | MISSING    |
pre_classifier.bias     | MISSING    |
classifier.bias         | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
```

Step 8: Define Training Settings

```
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=1
)
```

Step 9: Train Model

```
from transformers import AutoModelForSequenceClassification, Trainer

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=2
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_data,
    eval_dataset=test_data
)

trainer.train()
```

```
Loading weights: 100%                                    100/100 [00:00<00:00, 232.20it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_layer_norm.weight  | UNEXPECTED |
vocab_transform.weight   | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_layer_norm.bias    | UNEXPECTED |
classifier.weight        | MISSING    |
pre_classifier.weight    | MISSING    |
pre_classifier.bias      | MISSING    |
classifier.bias          | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
[250/250 01:58, Epoch 1/1]
```

Epoch	Training Loss	Validation Loss
1	No log	0.307920

```
Writing model shards: 100%                                    1/1 [00:01<00:00,   1.52s/it]
TrainOutput(global_step=250, training_loss=0.39828704833984374, metrics={'train_runtime': 120.7976, 'train_samples_per_second': 16.557, 'train_steps_per_second': 2.07, 'total_flos': 264934797312000.0, 'train_loss': 0.39828704833984374, 'epoch': 1.0})
```

Step 10: Evaluate & Test

```
results = trainer.evaluate()
print("Accuracy:", results["eval_accuracy"] if "eval_accuracy" in results else results)

# Test sentence
text = "This movie is fantastic!"

inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)
# Move input tensors to the same device as the model
inputs = {k: v.to(model.device) for k, v in inputs.items()}
outputs = model(**inputs)

prediction = torch.argmax(outputs.logits)
```